DocPilot

A Copilot-Inspired Multi-Model Document Intelligence Platform

Team NO: 38 · Domain: **Productivity** · S26CS6.401 — Project 3 Proposal

Description of the Use Case

Knowledge workers — students, researchers, legal professionals, and analysts — routinely manage large, multi-format document corpora: research papers, contracts, lecture notes, and reports. Existing tools such as ChatPDF or Adobe AI address only narrow slices of this problem: single-document Q&A with no persistent memory, no cross-document reasoning, and no intelligence about which model to invoke.

GitHub Copilot demonstrated that inserting an AI-native **pipeline directly into the editing workflow** — one that understands context, suggests actions, and executes them safely — fundamentally transforms how people work with content. **DocPilot** applies this same pipeline philosophy to documents. Rather than a chatbot that happens to read PDFs, DocPilot is a structured, multi-stage pipeline where every document operation (reading, editing, generation, cross-document synthesis) flows through context-aware, model-routed, and reversible execution stages. The architecture is the contribution, not merely the model call.

Stakeholders: Graduate researchers, law students, financial analysts, and educators working across heterogeneous document types.

Significance: The core engineering challenge is not model capability — it is the architecture *around* the model: how context is budgeted, how edits are isolated, how multiple models are composed, and how outputs are grounded to source documents.

Key Functionalities

Core Features

- **Grounded Document Q&A:** Query any uploaded document with answers citing exact page and paragraph sources via a RAG pipeline that retrieves, budgets, and grounds every response.
- **Scoped AI Editing with Diff View:** Request targeted rewrites; the system produces a structured diff with per-change accept/reject controls. The source document is never mutated until the user commits, backed by full undo via Command + Memento.
- **Cross-Document Synthesis:** Reason across multiple documents simultaneously — compare claims, detect contradictions, generate unified summaries — with every output claim linked to its source chunk.
- **RAG-Based Draft Generation:** Generate new reports, outlines, or summaries grounded in an uploaded corpus, with attribution of each generated paragraph to source material.
- **Intelligent Model Routing:** Route each task to the optimal model based on task type, cost, and latency — fast/cheap models for inline suggestions, large-context models for deep analysis, vision models for charts in PDFs.

User Interaction

A web interface presents a document editor alongside a context-aware assistant panel. Users upload documents (PDF, DOCX, PPTX, MD), issue natural language instructions, and receive inline suggestions analogous to Copilot's ghost-text experience. A multi-document workspace canvas supports cross-document queries.

Technical Stack

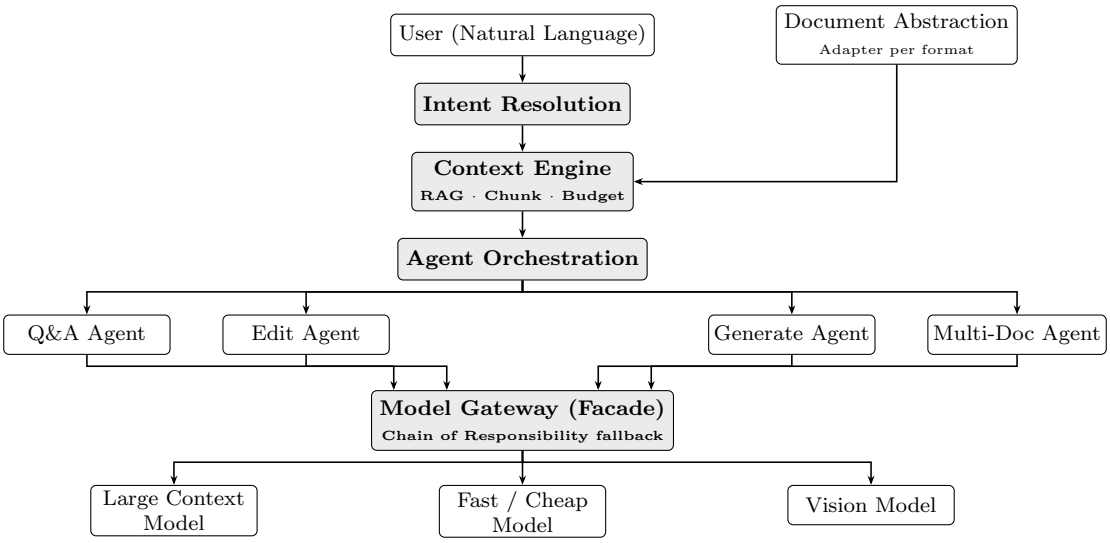
Frontend: React + TypeScript. **Backend:** Python (FastAPI). **Vector Store:** pgvector / Pinecone. **LLMs:** Claude 3, GPT-4o, Gemini 1.5 via a unified Model Gateway. **Parsing:** PyMuPDF, python-docx, Unstructured.io.

Design Patterns and Design Decisions

- **Pipeline Architecture:** Every operation (ingest → parse → chunk → embed → retrieve → generate → render) is a discrete, composable, replaceable stage — the Copilot request pipeline applied to documents.
- **Strategy:** Pluggable chunking strategies (fixed-size, semantic, structural) and model-selection policies are interchangeable at runtime without modifying pipeline logic.
- **Adapter:** Each document format (PDF, DOCX, MD, PPTX) has its own parser Adapter normalising output into a unified internal block tree (paragraphs, headings, tables, figures). All stages work against this abstraction only.
- **Command + Memento:** AI-proposed edits are encapsulated as reversible Command objects; document snapshots (Memento) enable multi-level undo and a complete audit trail.

- **Chain of Responsibility:** Model fallback chains — if the primary model is unavailable or over budget, the request passes to a fallback transparently. Also used for multi-stage query escalation.
- **Facade:** A single `ModelGateway` facade abstracts all LLM provider SDKs (Anthropic, OpenAI, Google) so agents never depend on a specific provider directly.
- **Observer:** Document change events propagate to the diff renderer, citation tracker, and session memory in real time without tight coupling.
- **Repository + CQRS:** The chunk and metadata stores are accessed through a Repository interface decoupled from the vector DB. Query paths (retrieval, Q&A) and write paths (edit, generate) are handled by distinct branches for independent scaling.

Architecture Diagram



Expected Time to Build a Prototype

The team of 5 will **architect the entire system** but **implement only 2–3 end-to-end use cases**:

(1) Grounded Document Q&A with source citations,

(2) Scoped AI editing with diff view and accept/reject, and

(3) model routing across at least two LLM providers. The remaining components (multi-doc synthesis, draft generation) will be fully designed but not implemented.

Phase	Duration	Activities
Research & Planning	3 days	Domain study, RAG framework evaluation, model API comparison, scope finalisation
Architecture Design	6 days	Component diagrams, pattern assignments, API contracts, DB schema, interface definitions
Development	2 weeks	*Stream A:* Ingestion + Context Engine. *Stream B:* Model Gateway + Agents. *Stream C:* Edit pipeline + Diff UI
Integration & Testing	4 days	End-to-end flow tests for 3 use cases, unit + integration tests across pipeline stages
Refinement	3 days	Bug fixes, edge-case hardening, documentation, demo preparation

Total: **4 weeks**. Three parallel development workstreams with daily syncs for interface integration.

Domain

Productivity. DocPilot directly addresses the challenge of literature synthesis, study material generation, and multi-source analysis in academic settings. The pipeline architecture is domain-agnostic by design — with different corpora and routing policies, the same system applies to legal document review, financial analysis, and clinical research — making the architectural contribution generalisable well beyond a single domain.